자료구조 및 알고리즘

Chapter 6. 스택

6.1 스택이란

6.2 리스트를 이용한 스택

6.3 연결 리스트를 이용한 스택

6.4 스택 응용

1. 스택이란

- **스택이란?**
  - 쌓고 빼는 자료 구조

그림 6-1 스택 개념의 일상 예

- **스택의 활용**
  - 키보드 입력을 하다가 백스페이스 키를 누르면 최근에 입력한 글자를 지운다.
  - 모든 편집기는 최근에 한 작업순으로 취소하는 기능이 있다. (ctrl + z)
  - 프로그램에서 함수 A가 함수 B를 호출하고 함수 B는 함수 C를 호출하는 호출 체인을 나중에 각 함수가 끝나면 돌아갈 때를 대비해서 호출 경로를 잘 관리해야 한다.

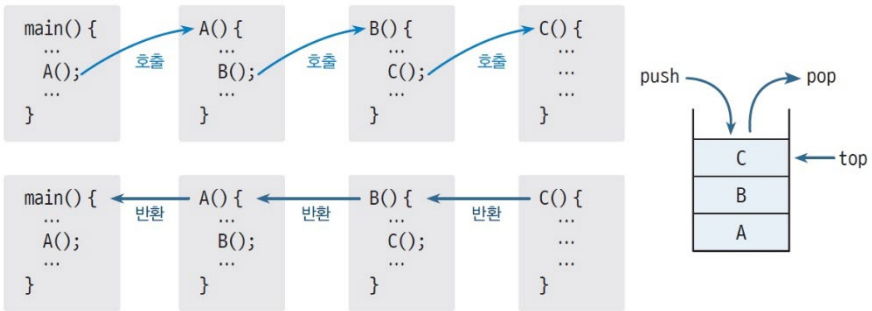


그림 6-7 함수 호출 시 스택 사용 예

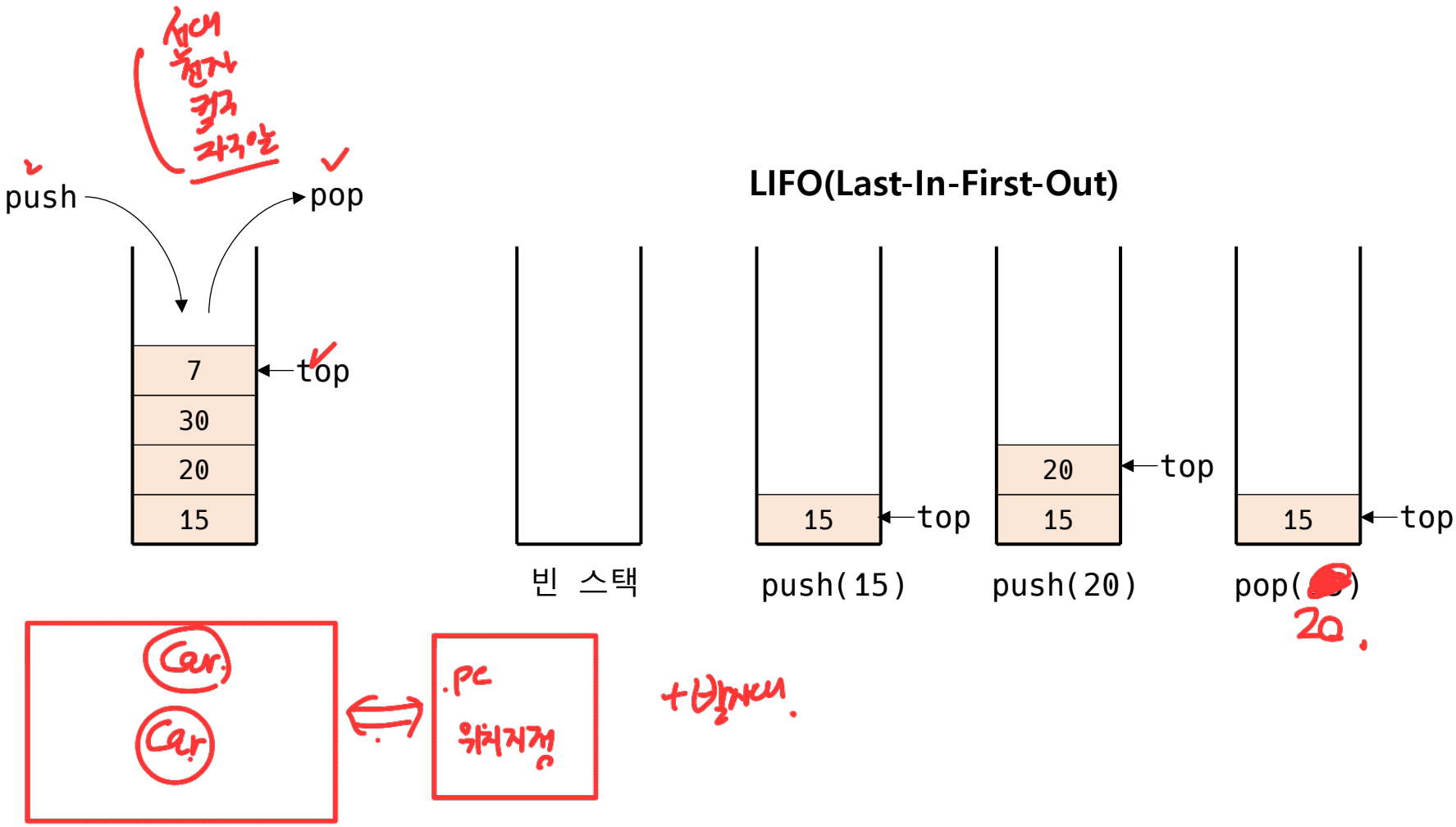
'←' in keyboard input line

e.g., abcd←←efgh←←←ij←km←

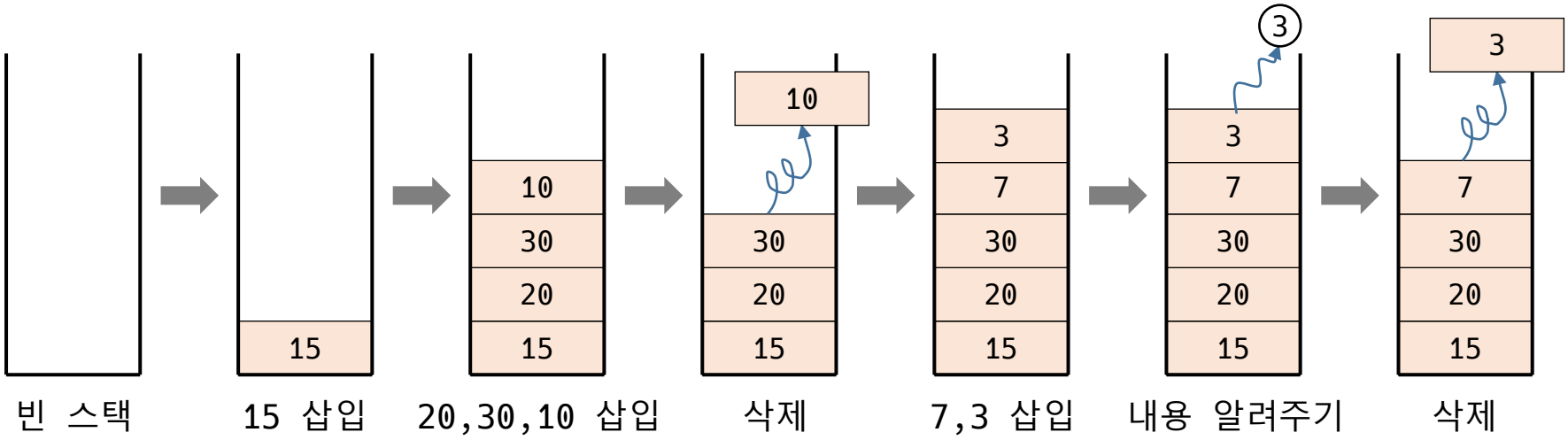
결과: abeik

한 문자를 읽어 '←' 이 아니면 저장하고
'←' 이면 **최근에 저장된** 문자를 제거한다.

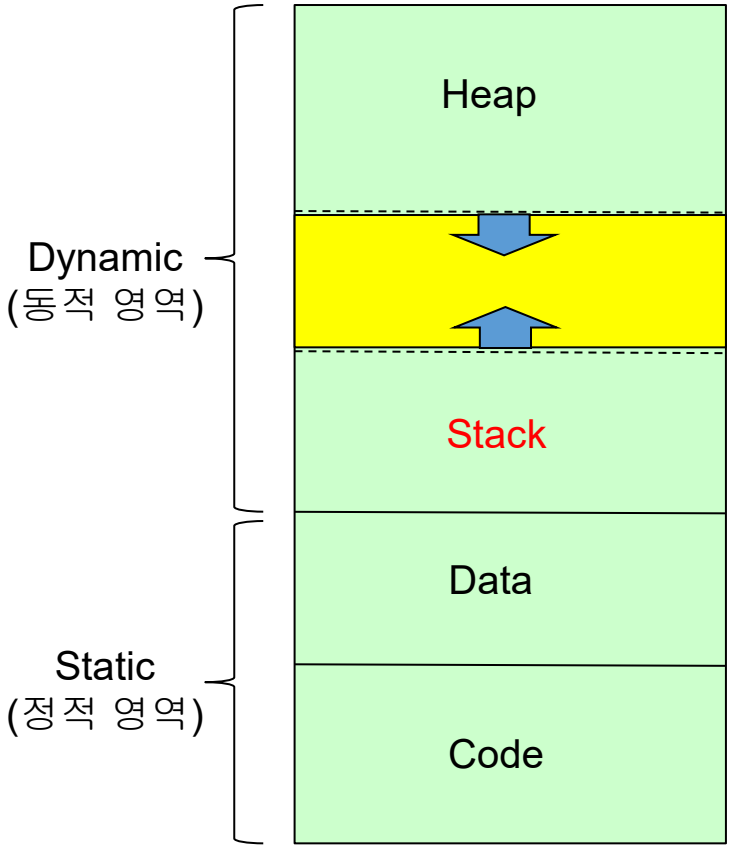
- 스택의 개념과 원리


push
pop
top
7
30
20
15
LIFO(Last-In-First-Out)
빈 스택
push(15)
push(20)
pop(
top
15
20
15
top
top
top

- **스택의 개념과 원리**
  - 넣을 때는 위로 쌓아 올리고, 뺄 때는 위에서부터 뺀다.
  - 맨 윗 원소의 내용만 볼 수 있다.



▪ 프로그램 수행과 스택 활용



수행중에 사용자가 할당받는 메모리
예: 자바의 new, C의 malloc(), 파이썬의 객체 생성

active 상태의 함수 관련 파라미터와 변수들

프로그램이 끝날 때까지 없어지지 않는 데이터.
전역 변수, 정적 변수

프로그램 수행 코드

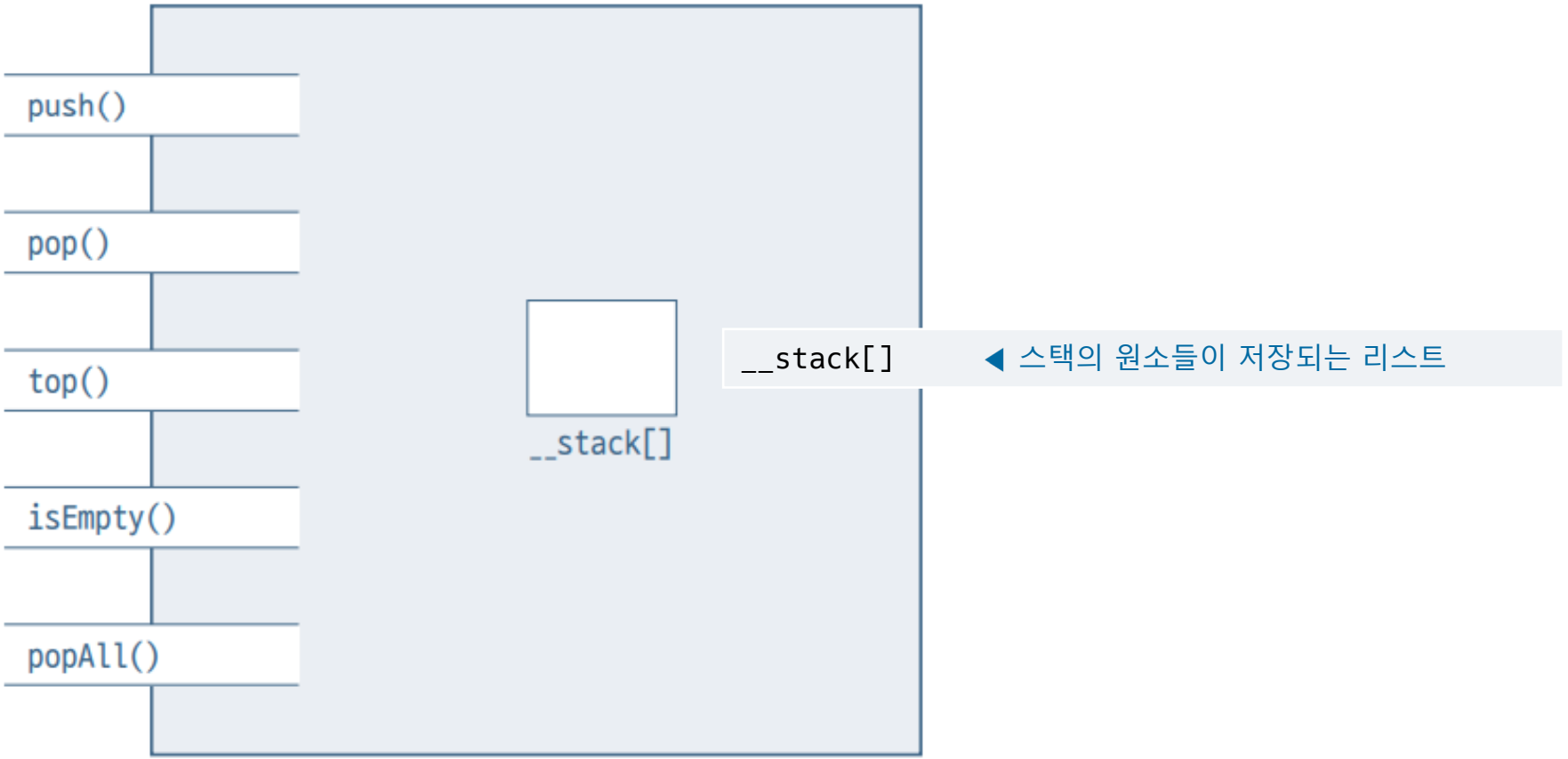
- **스택의 ADT**

맨 윗부분에 원소를 추가한다
맨 윗부분에 있는 원소를 알려준다
맨 윗부분에 있는 원소를 삭제하면서 알려준다
스택이 비어 있는지 확인한다
스택을 깨끗이 비운다

그림 6-8 ADT 스택

2. 리스트를 이용한 스택

- 리스트 스택의 객체 구조



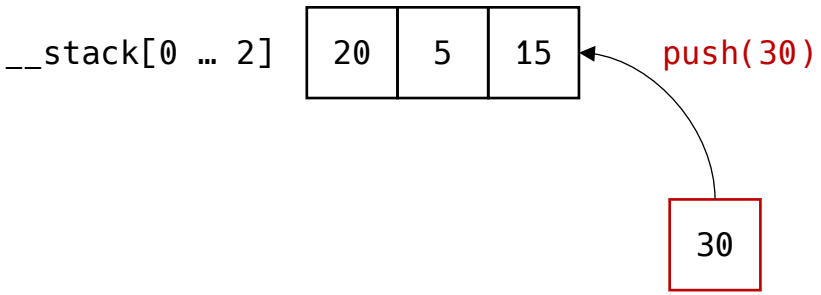
- **리스트 스택의 객체 구조**


push()
pop()
top()
isEmpty()
popAll()
push(x) ◀ 스택의 맨 위 원소에 x를 삽입한다.
pop( ) ◀ 스택의 맨 위에 있는 원소를 알려주고 삭제한다.
top( ) ◀ 스택의 맨 위에 있는 원소를 알려준다.
isEmpty( ) ◀ 스택이 비어있는지 알려준다.
popAll( ) ◀ 스택을 깨끗이 청소한다.

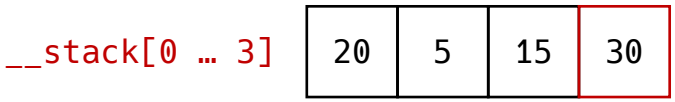
- **리스트 스택의 작업과 구현**
  - ListStack.py

```
class ListStack:
      def __init__(self):
            self.__stack = []
      def push(self, x): …
      def pop(self): …
      def top(self): …
      def isEmpty(self): …
      def popAll(self): …
```

- **리스트 스택의 작업과 구현**
  - 원소 삽입

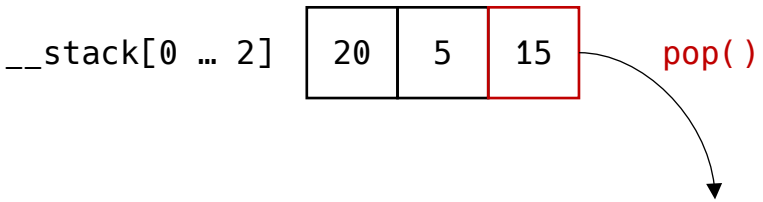


- **리스트 스택의 작업과 구현**
  - 원소 삽입




```
def push(self, x):
        self.__stack.append(x)
```

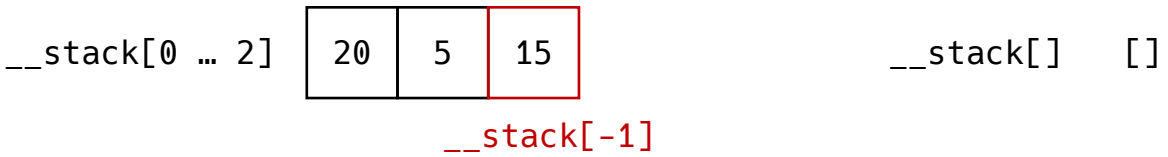
- **리스트 스택의 작업과 구현**
  - 원소 삭제



- **리스트 스택의 작업과 구현**
  - 원소 삭제

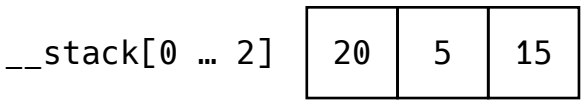

```
def pop(self):
        return self.__stack.pop()
```

- **리스트 스택의 작업과 구현**
  - 스택의 탑 원소 알려주기




```
def top(self):
        if self.isEmpty( ):
                return None
        else:
                return self.__stack[-1]
```

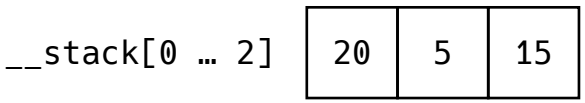
- **리스트 스택의 작업과 구현**
  - 스택이 비었는지 확인하기



bool(self.__stack) : True

```
def isEmpty(self) -> bool:
        return not bool(self.__stack)
```

- **리스트 스택의 작업과 구현**
  - 스택 비우기



- **리스트 스택의 작업과 구현**
  - 스택 비우기

```
__stack[]     []
```

```
def popAll(self):
        self.__stack = []
```

▪ 파이썬 구현

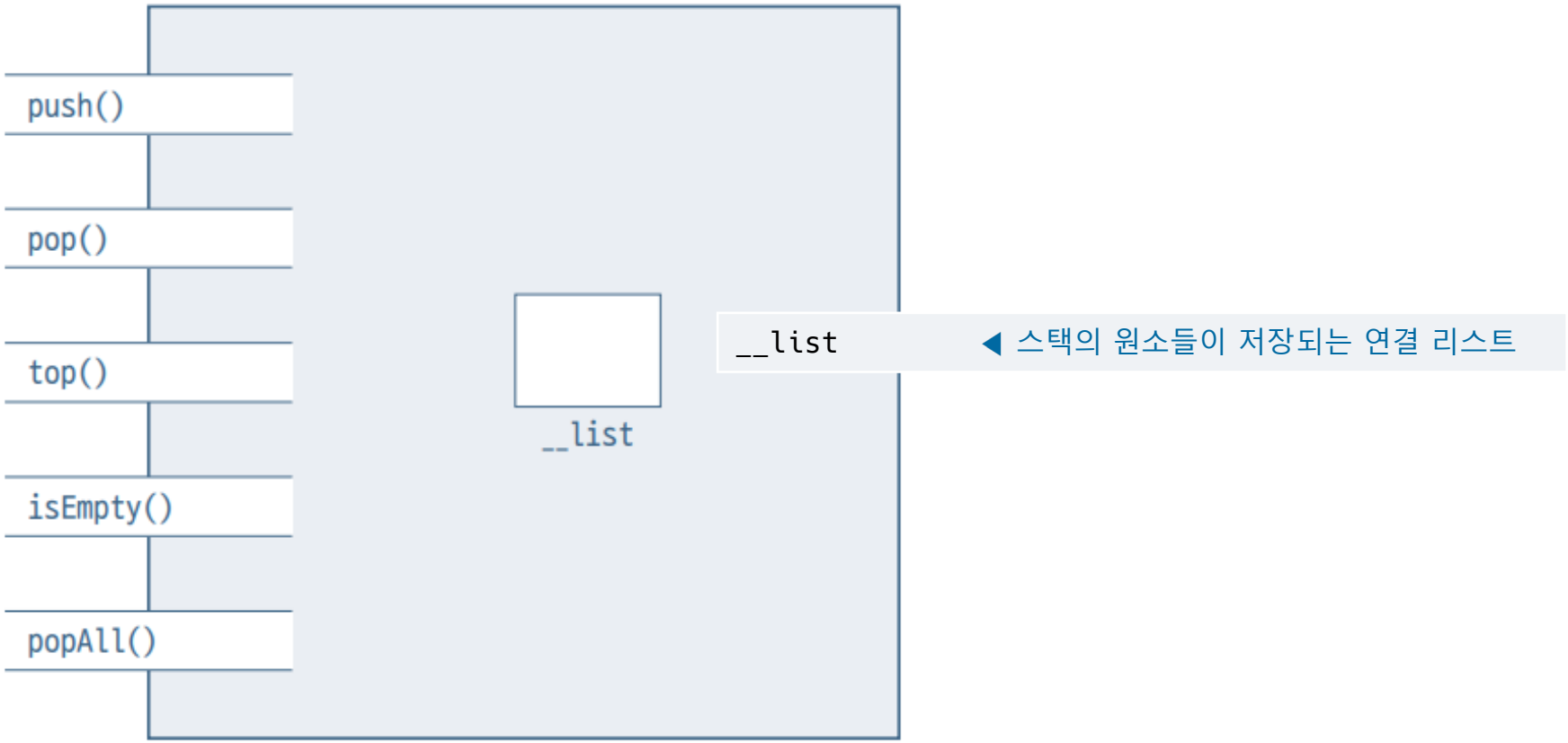
```
class ListStack:
    def __init__(self):
        self.__stack = []

    def push(self, x):
        self.__stack.append(x)

    def pop(self):
        return self.__stack.pop()

    def top(self):
        if self.isEmpty():
            return None
        else:
            return self.__stack[-1]

    def isEmpty(self) -> bool:
        return not bool(self.__stack)
        # 또는 return len(self.__stack) == 0

    def popAll(self):
        self.__stack.clear()

    def printStack(self):
        print("Stack from top:", end=' ')
        for i in range(len(self.__stack) -1, -1, -1):
            print(self.__stack[i], end=' ')
        print()
```

```
st1 = ListStack()
print(st1.top()) # No effect
st1.push(100)
st1.push(200)
print("Top is", st1.top())
st1.pop()
st1.push('Monday')
st1.printStack()
print('isEmpty?', st1.isEmpty())
```

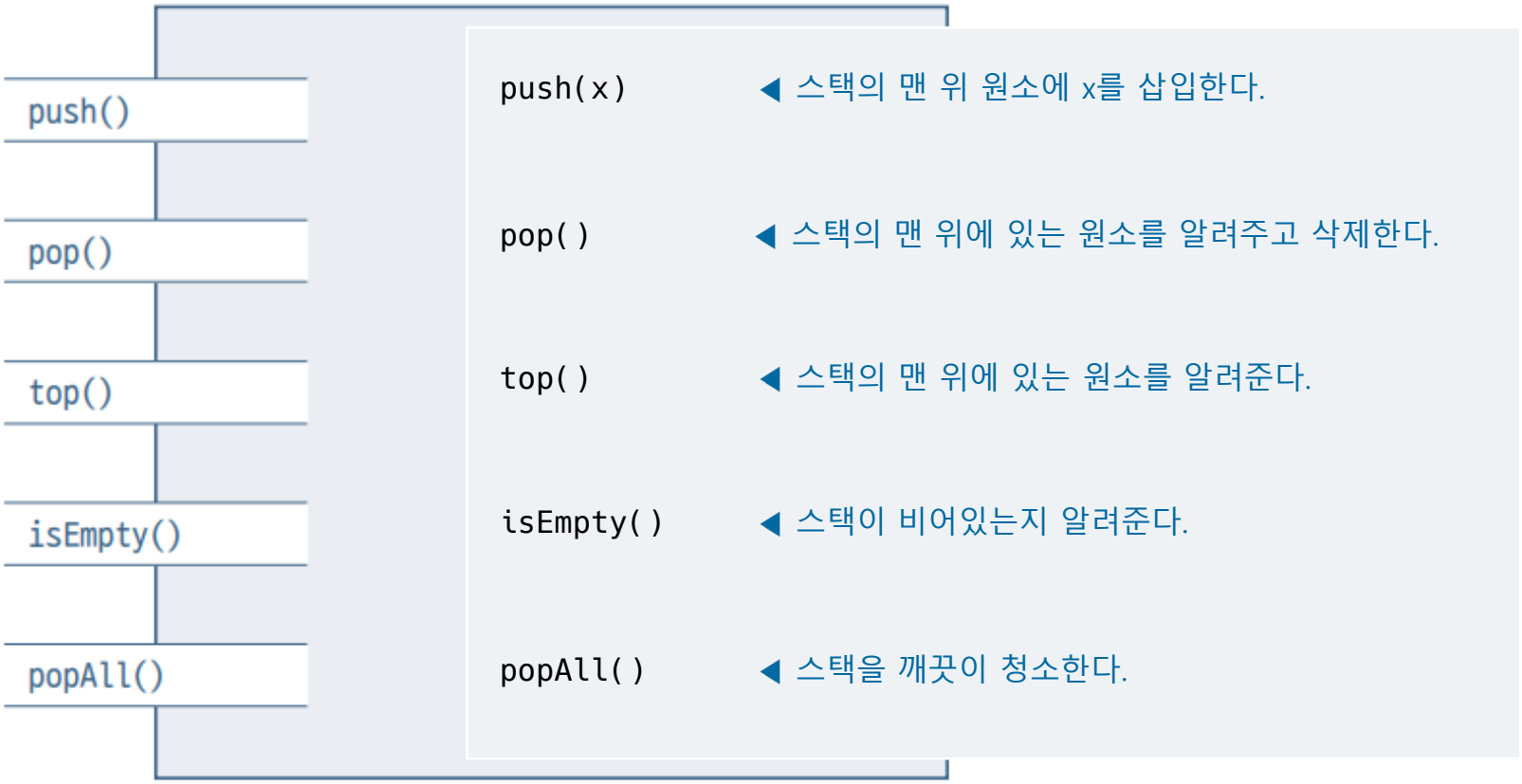
```
Top is 200
Stack from top: Monday 100
isEmpty? False
```

3. 연결 리스트를 이용한 스택

- 연결리스트 스택의 객체 구조



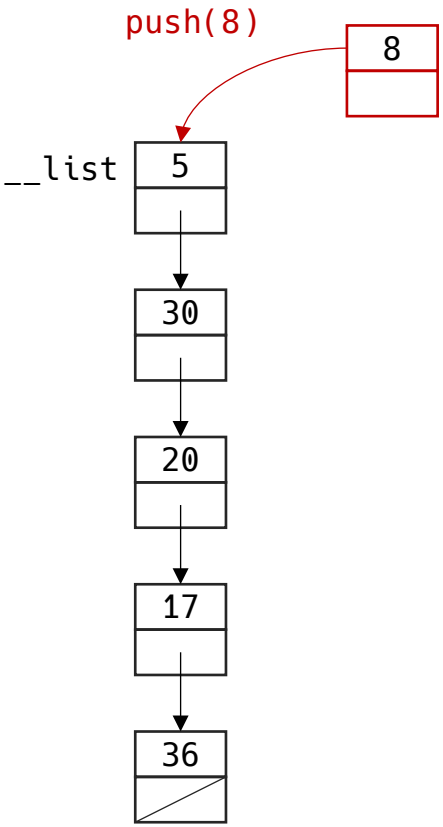
- **연결리스트 스택의 객체 구조**



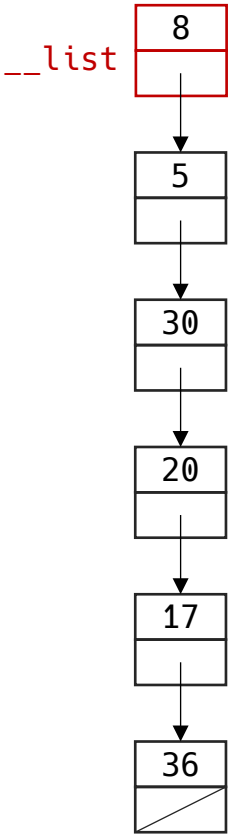
- **연결리스트 스택의 작업과 구현**
  - linkedStack.py

```
class LinkedStack:
      def __init__(self):
            self.__list = LinkedListBasic()
      def push(self, x): …
      def pop(self): …
      def top(self): …
      def isEmpty(self): …
      def popAll(self): …
```

- **연결리스트 스택의 작업과 구현**
  - 원소 삽입

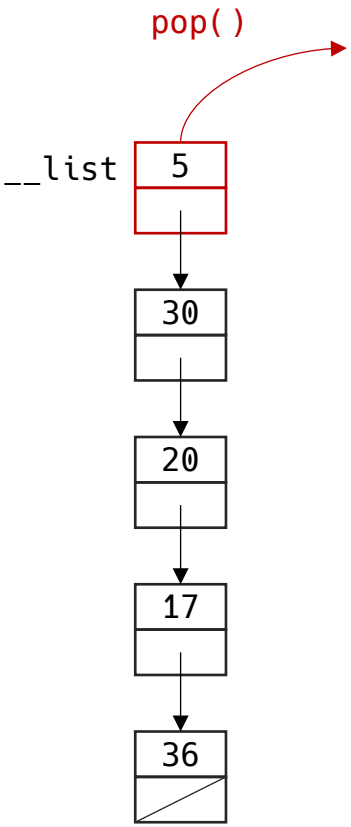


- **연결리스트 스택의 작업과 구현**
  - 원소 삽입

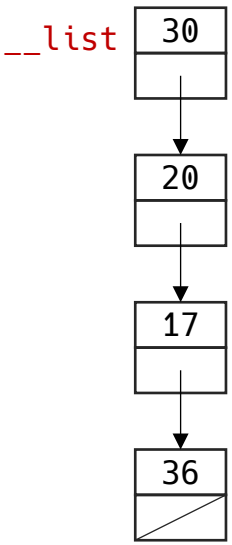



```
def push(self, x):
        self.__list.insert(0, x)
```

- **연결리스트 스택의 작업과 구현**
  - 원소 삭제



- **연결리스트 스택의 작업과 구현**
  - 원소 삭제




```
def pop(self):
        self.__list.pop(0)
```

- **연결리스트 스택의 작업과 구현**
  - 기타 작업

```
def top(self):
     if self.isEmpty( ):
          return None
     else:
          return self.__list.get(0)
```

```
def isEmpty(self):
     return self.__list.isEmpty( )
```

```
def popAll(self):
     self.__list.clear( )
```

▪ 파이썬 구현

```
class LinkedStack:
     def __init__(self):
          self.__list = LinkedListBasic()

     def push(self, newItem):
          self.__list.insert(0, newItem)

     def pop(self):
          return self.__list.pop(0)

     def top(self):
          return self.__list.get(0)

     def isEmpty(self) -> bool:
          return self.__list.isEmpty()

     def popAll(self):
          self.__list.clear()

     def printStack(self):
          print("Stack from top:", end=' ')
          for i in range(self.__list.size()):
               print(self.__list.get(i), end=' ')
          print()
```

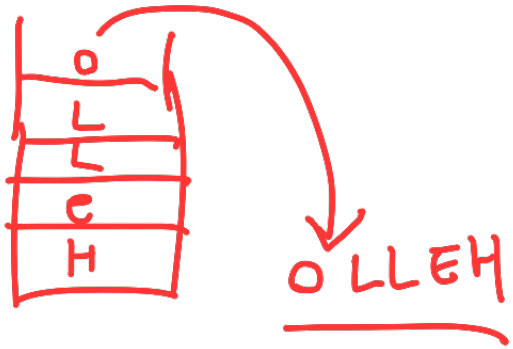
```
st1 = linkedStack()
st1.push(100)
st1.push(200)
print("Top is", st1.top())
st1.pop()
st1.push('Monday')
st1.printStack()
print('isEmpty?', st1.isEmpty())
```

```
Top is 200
Stack from top: Monday 100
isEmpty? False
```

3/31까지

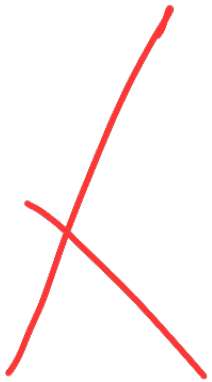
4. 스택 응용

- **문자열 뒤집기**
  - 주어진 문자열의 순서를 뒤집는 작업
  - 문자열이 끝날 때까지 push()를 통해 스택에 저장
  - 스택이 빌 때까지 pop()을 통해 문자를 하나씩 빼면서 출력




```
def reverse(str):
      st = ListStack()
      for i in range(len(str)):
            st.push(str[i])
      out = ""
      while not st.isEmpty():
            out += st.pop()
      return out

def main():
      input = "Test Seq 12345"
      answer = reverse(input)
      print("Input string: ", input)
      print("Reversed string: ", answer)

if __name__ == "__main__":
      main()
```

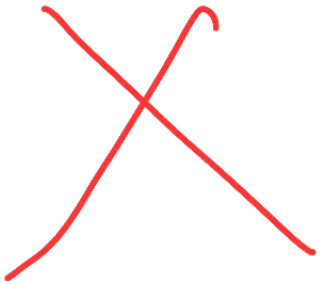
- **문자열 뒤집기**



```
def reverse(str):
      st = ListStack()
      for i in range(len(str)):
            st.push(str[i])
      out = ""
      while not st.isEmpty():
            out += st.pop()
      return out

def main():
      input = "Test Seq 12345"
      answer = reverse(input)
      print("Input string: ", input)
      print("Reversed string: ", answer)

if __name__ == "__main__":
      main()
```

- **문자열 뒤집기**

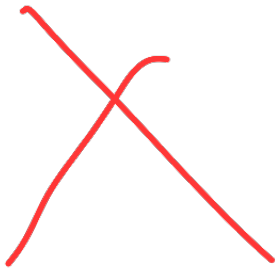
input = “Test Seq 12345”



```
def reverse(str):
      st = ListStack()
      for i in range(len(str)):
            st.push(str[i])
      out = ""
      while not st.isEmpty():
            out += st.pop()
      return out

def main():
      input = "Test Seq 12345"
      answer = reverse(input)
      print("Input string: ", input)
      print("Reversed string: ", answer)

if __name__ == "__main__":
      main()
```

- **문자열 뒤집기**

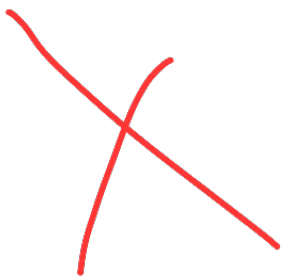
input = “Test Seq 12345”



```
def reverse(str):
      st = ListStack()
      for i in range(len(str)):
            st.push(str[i])
      out = ""
      while not st.isEmpty():
            out += st.pop()
      return out

def main():
      input = "Test Seq 12345"
      answer = reverse(input)
      print("Input string: ", input)
      print("Reversed string: ", answer)

if __name__ == "__main__":
      main()
```

- **문자열 뒤집기**

```
input = “Test Seq 12345”

st    []
```



```
def reverse(str):
      st = ListStack()
      for i in range(len(str)):
            st.push(str[i])
      out = ""
      while not st.isEmpty():
            out += st.pop()
      return out

def main():
      input = "Test Seq 12345"
      answer = reverse(input)
      print("Input string: ", input)
      print("Reversed string: ", answer)

if __name__ == "__main__":
      main()
```

- **문자열 뒤집기**

input = “Test Seq 12345”

st

T	e	s	t		S	e	q		1	2	3	4	5

```
def reverse(str):
      st = ListStack()
      for i in range(len(str)):
            st.push(str[i])
      out = ""
      while not st.isEmpty():
            out += st.pop()
      return out

def main():
      input = "Test Seq 12345"
      answer = reverse(input)
      print("Input string: ", input)
      print("Reversed string: ", answer)

if __name__ == "__main__":
      main()
```

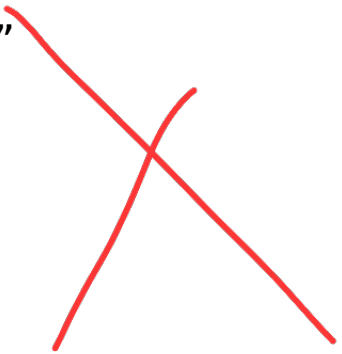
- **문자열 뒤집기**

input = “Test Seq 12345”

st

T	e	s	t		S	e	q		1	2	3	4	5

out = “”

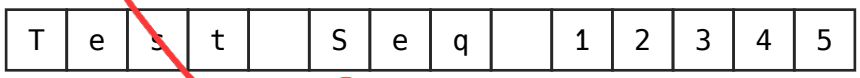


```
def reverse(str):
      st = ListStack()
      for i in range(len(str)):
            st.push(str[i])
      out = ""
      while not st.isEmpty():
            out += st.pop()
      return out

def main():
      input = "Test Seq 12345"
      answer = reverse(input)
      print("Input string: ", input)
      print("Reversed string: ", answer)

if __name__ == "__main__":
      main()
```

- **문자열 뒤집기**

input = “Test Seq 12345”

st



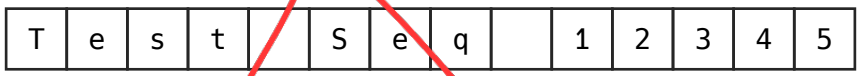
out = “54321 qeS tseT”

```
def reverse(str):
      st = ListStack()
      for i in range(len(str)):
            st.push(str[i])
      out = ""
      while not st.isEmpty():
            out += st.pop()
      return out

def main():
      input = "Test Seq 12345"
      answer = reverse(input)
      print("Input string: ", input)
      print("Reversed string: ", answer)

if __name__ == "__main__":
      main()
```

- 문자열 뒤집기

input = “Test Seq 12345”

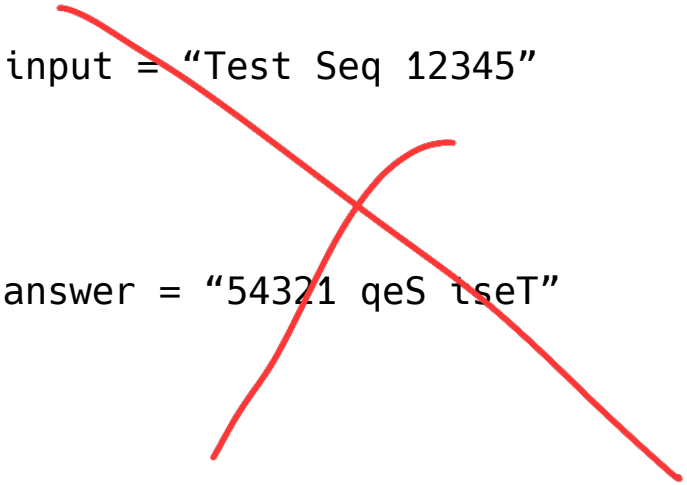
st



out = “54321 qeS tseT”

```
def reverse(str):
      st = ListStack()
      for i in range(len(str)):
            st.push(str[i])
      out = ""
      while not st.isEmpty():
            out += st.pop()
      return out

def main():
      input = "Test Seq 12345"
      answer = reverse(input)
      print("Input string: ", input)
      print("Reversed string: ", answer)

if __name__ == "__main__":
      main()
```

- **문자열 뒤집기**




```
def reverse(str):
      st = ListStack()
      for i in range(len(str)):
            st.push(str[i])
      out = ""
      while not st.isEmpty():
            out += st.pop()
      return out

def main():
      input = "Test Seq 12345"
      answer = reverse(input)
      print("Input string: ", input)
      print("Reversed string: ", answer)

if __name__ == "__main__":
      main()
```

- **문자열 뒤집기**

input = “Test Seq 12345”

answer = “54321 qeS tseT”

```
Input string: Test Seq 12345
Reversed string: 54321 qeS tseT
```

```
def reverse(str):
      st = ListStack()
      for i in range(len(str)):
            st.push(str[i])
      out = ""
      while not st.isEmpty():
            out += st.pop()
      return out

def main():
      input = "Test Seq 12345"
      answer = reverse(input)
      print("Input string: ", input)
      print("Reversed string: ", answer)

if __name__ == "__main__":
      main()
```

넣고

뺀다

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

postfix = “700 3 47 + 6 * - 4 /”

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

```
postfix = “700 3 47 + 6 * - 4 /”
```

```
Input string: 700 3 47 + 6 * - 4 /
```

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

```
postfix = “700 3 47 + 6 * - 4 /”
```

```
Input string: 700 3 47 + 6 * - 4 /
```

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

```
p = “700 3 47 + 6 * - 4 /”

s []
```

▪ Postfix 계산

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

```
p = “700 3 47 + 6 * - 4 /”

s []

digitPreviously = False
```

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
            or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
            '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

```
p = “700 3 47 + 6 * - 4 /”

s []

digitPreviously = False

i = 0
```

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

```
p = “700 3 47 + 6 * - 4 /”

s []

digitPreviously = False

i = 0

ch = ‘7’
```

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s []

digitPreviously = False

i = 0

ch = ‘7’

ch.isdigit() = True

▪ Postfix 계산

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s []

digitPreviously = False

i = 0

ch = ‘7’

▪ Postfix 계산

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s 7

digitPreviously = False

i = 0

ch = ‘7’

ord(ch) - ord('0’) = 7

▪ Postfix 계산

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s | 7 |

digitPreviously = True

i = 0

ch = ‘7’

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s | 7 |

digitPreviously = True

i = 1

ch = ‘0’

▪ Postfix 계산

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s [7]

digitPreviously = True

i = 1

ch = ‘0’

ch.isdigit() = True

▪ Postfix 계산

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s | 7 |

digitPreviously = True

i = 1

ch = ‘0’

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

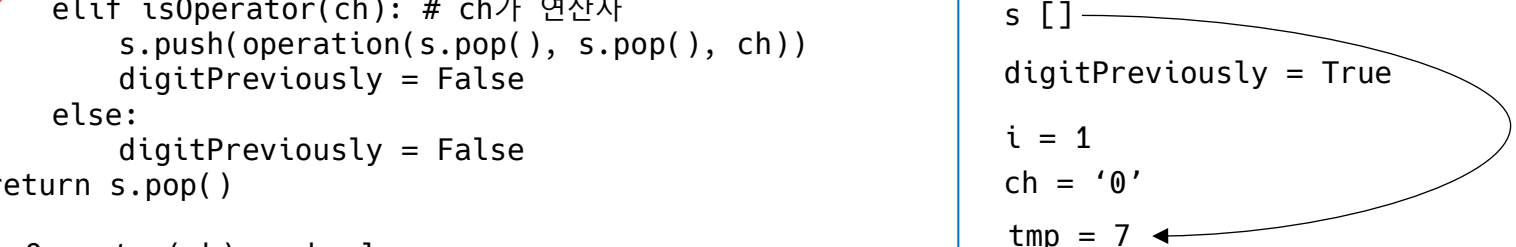
s []

digitPreviously = True

i = 1

ch = ‘0’

tmp = 7



▪ Postfix 계산

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s []

digitPreviously = True

i = 1

ch = ‘0’

tmp = 10 * 7 + 0 = 70

▪ Postfix 계산

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s | 70 |

digitPreviously = True

i = 1

ch = ‘0’

tmp = 70

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s | 700 |

digitPreviously = True

i = 2

ch = ‘0’

tmp = 700

Postfix 계산

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s | 700 |

digitPreviously = True

i = 3

ch = ‘ ’

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s | 700 |

digitPreviously = True

i = 3

ch = ‘ ’

ch.isdigit() = False

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s 700

digitPreviously = True

i = 3

ch = ‘ ’

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
            or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
            '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s | 700 |

digitPreviously = True

i = 3

ch = ‘ ’

isOperator(ch) = False

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
            or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
            '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s [700]

digitPreviously = False

i = 3

ch = ‘ ’

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s | 700 |

digitPreviously = False

i = 4

ch = ‘3’

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s | 700 |

digitPreviously = False

i = 4

ch = ‘3’

ch.isdigit() = True

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s 700

digitPreviously = False

i = 4

ch = ‘3’

▪ Postfix 계산

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s | 700 | 3 |

digitPreviously = False

i = 4

ch = ‘3’

ord(ch) - ord('0’) = 3

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s

700	3

digitPreviously = True

i = 4

ch = ‘3’

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s

700	3	47

digitPreviously = False

i = 8

ch = ‘ ’

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s

700	3	47

digitPreviously = False

i = 9

ch = ‘+’

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s

700	3	47

digitPreviously = False

i = 9

ch = ‘+’

ch.isdigit() = False

▪ Postfix 계산

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s

700	3	47

digitPreviously = False

i = 9

ch = ‘+’

isOperator(ch) = True

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s

700	3	47

digitPreviously = False

i = 9

ch = ‘+’

▪ Postfix 계산

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s

700	3	47

digitPreviously = False

i = 9

ch = ‘+’

▪ Postfix 계산

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s

700	3

digitPreviously = False

i = 9

ch = ‘+’

s.pop() = 47

▪ Postfix 계산

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s 700

digitPreviously = False

i = 9

ch = ‘+’

s.pop() = 47

s.pop() = 3

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s | 700 |

digitPreviously = False

i = 9

ch = ‘+’

s.pop() = 47

s.pop() = 3

operation(47, 3, ‘+’) = 50

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s | 700 | 50 |

digitPreviously = False

i = 9

ch = ‘+’

s.pop() = 47

s.pop() = 3

operation(47, 3, ‘+’) = 50

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s

700	50

digitPreviously = False

i = 9

ch = ‘+’

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s

700	50	6

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s

700	300

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s | 400 |

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s

400	4

▪ Postfix 계산

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

p = “700 3 47 + 6 * - 4 /”

s | 100 |

▪ Postfix 계산

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

```
p = “700 3 47 + 6 * - 4 /”

s []

s.pop() = 100
```

- **Postfix 계산**

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

```
postfix = “700 3 47 + 6 * - 4 /”

answer = 100
```

```
Input string: 700 3 47 + 6 * - 4 /
```

▪ Postfix 계산

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
                or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
                '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix);
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

```
postfix = “700 3 47 + 6 * - 4 /”

answer = 100
```

```
Input string: 700 3 47 + 6 * - 4 /
Answer: 100
```